OCSF ThreatPulse

SOC Analyst Dashboard

Complete Website Flow & Architecture Document

Prepared for: SOC Engineering Team

Project: OCSF-Based Hybrid Threat Detection Pipeline

Date: June 08, 2026

Classification: INTERNAL USE ONLY

Property	Detail
Total Pages	5 Application Pages
Tech Stack	React 18+ TypeScript + Vite + Tailwind CSS
State Management	TanStack Query + Zustand
UI Components	shadcn/ui + Recharts + Lucide React
Backend Integration	FastAPI via Axios (REST)
Theme	Dark Mode - SOC Optimized *(Note: Core system architecture optimized for dark-SOC deployment)*
Auto Refresh	Every 30 seconds via TanStack Query

1. Project Overview & Purpose

The OCSF Threat Pulse SOC Analyst Dashboard is the front-end command centre for the OCSF-Based Hybrid Threat Detection Pipeline. It gives security analysts a single, unified interface to monitor network threat detections, explore anomalies, manage the Dead Letter Queue, and assess overall pipeline health all in real time.

The backend pipeline processes raw network event logs through three sequential defense layers: a Volumetric Statistical Filter (Layer 1), a Random Forest Classifier with SHAP explainability (Layer 2), and a PyTorch LSTM sequential tracker (Layer 3). This dashboard surfaces every output of that pipeline in an optimized user interface.

Why This Dashboard Is Critical

- Approximately 68% of threats are missed by basic firewall-level detection - this dashboard exposes what lies beneath.
- SHAP feature attributions from Layer 2 let analysts understand exactly why an event was flagged, not just that it was.
- The LSTM sequential tracker detects slow, multi-hop attacks that span time - the Anomaly Explorer page visualizes this.
- The DLQ Monitor gives operations teams visibility into data reliability failures that would otherwise be invisible.
- 30-second auto-refresh ensures analysts always see the current threat landscape without manual intervention.

Backend API Endpoints Consumed

Endpoint	Method	Used By	Purpose
`/api/v1/health`	GET	Dashboard, Navbar	Redis + DB + Pipeline status
`/api/v1/detect`	POST	Alerts, Anomalies	Main pipeline threat output
`/api/v1/ingest`	POST	Internal	Raw log ingestion
`/api/v1/dlq/requeue`	POST	DLQ Monitor	Trigger manual DLQ requeue
`/metrics`	GET	Dashboard	Prometheus queue + latency metrics

2. Route Structure & Navigation Architecture

The application uses React Router v6 for client-side routing. There are five distinct routes. The Home page (/) is a public-facing landing page with its own top navbar. All inner pages (/dashboard, /alerts, /anomalies, /dlq) share a persistent left sidebar for navigation between operational views.

Complete Route Map

Route	Page Name	Component File & Navigation Element
`/`	Home Landing Page	Top Navbar | `pages/Home.tsx`
`/dashboard`	Dashboard Overview	Left Sidebar | `pages/Dashboard.tsx`
`/alerts`	Threat Alerts	Left Sidebar | `pages/Alerts.tsx`
`/anomalies`	Anomaly Explorer	Left Sidebar | `pages/Anomalies.tsx`
`/dlq`	DLQ Monitor	Left Sidebar | `pages/DLQ.tsx`

Navigation Flow Diagram

```
[ Home Page (/) ]  <-- public landing, own top navbar
       |
       ▼ (Enter Dashboard Button)
┌─────────────────────────────────────────────────────────┐
│                 LEFT SIDEBAR (Persistent)               │
│                                                         │
│  [icon] Dashboard Overview     -->  /dashboard          │
│  [icon] Threat Alerts          -->  /alerts             │
│  [icon] Anomaly Explorer       -->  /anomalies          │
│  [icon] DLQ Monitor            -->  /dlq                │
│                                                         │
│  [logo] <-- Returns to Home                             │
└─────────────────────────────────────────────────────────┘
```

The sidebar collapses to icon-only mode on mobile devices. The collapsed/expanded state is managed globally via Zustand so it persists when navigating between pages. The active route is highlighted using React Router NavLink's active class.

Global State (Zustand Store)

State Key	Type	Used By	Purpose
`activeIPFilter`	string | null	Anomaly Explorer	Filter all charts by source IP
`selectedAlert`	AlertObject | null	Threat Alerts	Controls Drawer open/close + content
`sidebarCollapsed`	boolean	All inner pages	Sidebar expand/collapse on mobile

3. Page Flow - Home Landing Page (/)

The Home page is the entry point of the application. Its purpose is to communicate the project's value, explain the 3-layer detection pipeline, and direct analysts into the operational dashboard. It is inspired by dark intelligence platform aesthetics: strong hero section, animated background, scrollable feature sections, and clear CTAs.

- **Section 1 - Sticky Navbar:** Transparent background on page load, transitions to solid dark on scroll. Left side features the OCSF Threat Pulse logo with shield icon (Lucide). Right side hosts navigation links (Dashboard, Alerts, Anomalies, DLQ) and an [Enter Dashboard] CTA button. On mobile, a hamburger menu collapses nav links.
- **Section 2 - Hero:** Dark background with animated moving network graph particle lines (CSS/canvas animation). Main headline: *'Real-Time Threat Detection Powered by OCSF'*. Subtitle explaining the 3-layer AI pipeline in one sentence. Two CTA buttons: [Enter Dashboard →] and [Learn More (anchor scroll)]. Live stat ticker below the CTAs, auto-refreshing from `/api/v1/health` and `/metrics`: Events Processed | Threats Caught | DLQ Size | Uptime.
- **Section 3 - Why This Matters:** Short editorial paragraph: *'Every second, thousands of network events go unanalyzed. Most SIEM tools react. Threat Pulse predicts.'* Features three animated count-up stat cards that trigger on scroll into view:

Stat	Value	Label / Benchmark
Threats missed by basic firewalls	~68%	Industry benchmark
Detection pipeline layers	3-Layer	***Stats + RF + LSTM***
Detection latency per event	<50ms	CPU-optimized inference

- **Section 4 - How the Pipeline Works:** Horizontal interactive step-by-step flow diagram detailing the pipeline path:

> Raw Logs → OCSF Normalize → Layer 1 Statistical Filter → Layer 2 Random Forest + SHAP → Layer 3 LSTM → Threat Alert Stored

  Clicking each step reveals a tooltip with a one-line description.

- **Section 5 - What You Can Do (Feature Cards):** Four cards, one per inner page, each with an icon, title, description, and an [Open →] button:

Card Name	Icon	Description & Route Target
Dashboard	BarChart2	Live system health and alert volume overview (`/dashboard`)
Threat Alerts	AlertTriangle	Sortable threat table with SHAP explanations (`/alerts`)
Anomaly Explorer	Search	Scatter, Entropy, and EWMA charts per IP (`/anomalies`)
DLQ Monitor	Skull	Dead letter queue admin and requeue controls (`/dlq`)

- **Section 6 - Tech Stack Strip:** Horizontal logo row highlighting production-grade open standards: OCSF | FastAPI | Redis | PostgreSQL | PyTorch | Scikit-Learn | SHAP | Prometheus. Greyscale icons where hover reveals color signals.
- **Section 7 - Footer:** OCSF ThreatPulse branding, 'Built for SOC Teams' tagline. Links out to GitHub repository, API Docs, and a back-to-top anchor.

4. Page Flow - Dashboard Overview (/dashboard)

The Dashboard is the first page an analyst lands on after clicking Enter Dashboard. It gives a full-picture health and volume view of the pipeline at a glance.

User Journey

- TanStack Query fires `GET /api/v1/health` and `GET /metrics` on mount.
- Health badges render immediately: Redis OK/FAIL, Database OK/FAIL, Pipeline OK/FAIL.
- Four shadcn Cards render: Total Alerts Today | Active Threats | DLQ Size | Events Processed.
- Recharts AreaChart renders alert volume for the last 60 minutes, refetching every 30 seconds silently.
- Recharts BarChart shows Layer 1 dropped / Layer 2 flagged / Layer 3 confirmed event counts.
- All data is sourced via an Axios instance configured from the `VITE_API_BASE_URL` environment variable.

5. Page Flow - Threat Alerts (/alerts)

The Alerts page is the primary investigation surface for SOC analysts. Every threat detected by any layer of the pipeline appears here in a filterable, sortable table.

User Journey

- TanStack Query fetches alert data on mount and refetches every 30 seconds.
- Filter bar at the top includes: IP search input (shadcn Input), Protocol selector (shadcn Select), and Layer selector (shadcn Select).
- All filters update the Zustand store and re-filter the table client-side without making additional API calls.

- A shadcn Table renders with columns: Timestamp | Src IP | Dst IP | Protocol | Layer | Confidence | Top SHAP Feature.
- Layer severity is shown as a color-coded shadcn Badge: **Layer 1 (Yellow)**, **Layer 2 (Orange)**, or **Layer 3 (Red)**.
- Analyst clicks a table row → Zustand stores `selectedAlert` → shadcn Drawer slides in from the right.

Detail Drawer Contents

Drawer Section	Content Details
OCSF Event Fields	src_ip, dst_ip, protocol, port, bytes, packets, duration, timestamp
SHAP Attributions	Recharts HorizontalBarChart showing top feature contributions
Layer Verdict	Which layer flagged it and the confidence score
Raw JSON Toggle	Expandable raw OCSF event payload for deep inspection

6. Page Flow - Anomaly Explorer (/anomalies)

The Anomaly Explorer is the analytical deep-dive page. It exposes the raw statistical signals computed by Layer 1 of the pipeline - Z-Score, Shannon Entropy, EWMA - visualized per source IP over time.

User Journey

- IP Selector (shadcn Select) renders at top; default is 'All IPs'.
- Analyst selects an IP → Zustand `activeIPFilter` updates → all three charts re-filter simultaneously.
- **Recharts ScatterChart:** Anomaly score per source IP over time. IPs above Layer 1 threshold are shown as RED dots, normal IPs as grey.
- **Recharts LineChart:** Shannon Entropy of destination ports over the rolling window for the selected IP.
- **Recharts AreaChart:** EWMA flow volume byte trend per host over time.
- All charts share the same time range and IP filter, ensuring the analyst gets a correlated view of all three signals.

7. Page Flow - DLQ Monitor (/dlq)

The DLQ Monitor page gives operations engineers and senior analysts visibility into database write failures and control over the recovery process.

User Journey

- TanStack Query fetches DLQ data on mount, refetching every 30 seconds.

- shadcn Card at top shows current DLQ depth as a large counter.
- shadcn Table lists DLQ items: Retry Count | First Failed At | Last Failed At | Failure Reason.
- Analyst clicks the [Requeue] button (shadcn Button) → Axios `POST /api/v1/dlq/requeue` fires.
- On response, a shadcn Toast notification appears showing the Requeued count and Discarded count.
- The TanStack Query cache for DLQ data is immediately invalidated, refreshing the table depth instantly.

DLQ Requeue Response Toast Examples

Scenario	Toast Message	Toast Variant
Successful requeue	"Requeued 7 alerts. Discarded 2 (max retries)."	Success (green)
All discarded	"0 requeued. 10 discarded as permanent failures."	Warning (yellow)
API failure	"Requeue failed. Check pipeline health."	Error (red)
Empty DLQ	"DLQ is empty. Nothing to requeue."	Info (blue)

8. End-to-End Data Flow

This section describes how data moves from the environment configuration through the frontend stack into the user interface elements:

```
[.env File]  VITE_API_BASE_URL = http://localhost:8000
    │
    ▼
[src/api/] (Axios instance with base URL interceptors)
    ├── healthApi.ts  --> GET /api/v1/health
    ├── alertsApi.ts  --> POST /api/v1/detect
    ├── metricsApi.ts --> GET /metrics
    └── dlqApi.ts     --> POST /api/v1/dlq/requeue
    │
    ▼
[TanStack Query] (Handles caching & global refetchInterval: 30000ms)
    │
    ▼
[Zustand Store]  (Manages client state: activeIPFilter, selectedAlert,
sidebarCollapsed)
    │
    ▼
[React Router v6 Routes]
    ├── /            --> Home.tsx
    ├── /dashboard   --> Dashboard.tsx
    ├── /alerts      --> Alerts.tsx
    ├── /anomalies   --> Anomalies.tsx
    └── /dlq         --> DLQ.tsx
    │
    ▼
[shadcn/ui & Recharts] (Renders final elements into dark/light optimized visual
components)
    │
    ▼
[Analyst Screen] (Displays active live threat telemetry grids)
```

9. Frontend File Structure

```
frontend/src/
├── api/
│   ├── alertsApi.ts
│   ├── metricsApi.ts
│   ├── dlqApi.ts
│   ├── healthApi.ts
│   └── axiosInstance.ts
├── components/
│   ├── Sidebar.tsx
│   ├── Navbar.tsx
│   ├── AlertDrawer.tsx
│   ├── HealthBadge.tsx
│   └── LayerBadge.tsx
├── pages/
│   ├── Home.tsx
│   ├── Dashboard.tsx
│   ├── Alerts.tsx
│   ├── Anomalies.tsx
│   └── DLQ.tsx
├── store/
│   └── useAppStore.ts (Zustand)
├── mocks/
│   ├── alertsMock.ts
│   ├── metricsMock.ts
│   └── dlqMock.ts
├── App.tsx
└── main.tsx

Root Configs:
├── vite.config.ts
├── tailwind.config.ts
└── tsconfig.json
```

10. Tech Stack Summary

Library	Version	Role in Project
React	18	Component framework
TypeScript	5.x	Type safety across all components and API calls
Vite	5.x	Build tool and dev server with HMR
Tailwind CSS	v3	Utility-first application layout and responsive styling
shadcn/ui	latest	Accessible UI primitives: Table, Card, Badge, Drawer, Toast
Recharts	2.x	Data visualization: AreaChart, BarChart, ScatterChart, LineChart
TanStack Query	v5	Server state management, caching, and 30s auto-refetch
React Router	v6	Client-side routing and NavLink active state highlighting
Axios	1.x	HTTP client with centralized base URL configuration + interceptors
Zustand	4.x	Lightweight global state for cross-component filters and UI states
date-fns	3.x	Timestamp formatting and rolling window window calculations
Lucide React	0.383	Icon library fully integrated with shadcn/ui primitives